Formal Embedding Design

Assignment 2 — Vector Embedding for Capability Composition

Mathematical specification • representation • compatibility • composition

Executive design summary

The proposed Structured Capability Embedding (SCE) is a deliberately structured vector representation for formally specified states, goals, and executable capabilities. Instead of treating a capability as undifferentiated text, the representation preserves the fields that determine whether operations can actually compose: inputs, outputs, preconditions, effects, constraints, resources, and operational attributes.

Design principle. Functional resemblance and composability are different relations. SCE therefore uses ordinary cosine similarity for resemblance while using a separate *directed* compatibility score for composition.

The assignment defines a capability as a reusable operation characterized by what it requires, produces, changes, and the conditions under which it operates. The design below follows that formal framing.

1. Formal application model

The application is represented as

$$A = (S, C, S_I, G, R, K)$$

S — state space **C** — available capabilities S_I — initial state **G** — goal specification **R** — resources **K** — global constraints and policies.

A state is a set of variable/value pairs, while a goal is a set of desired conditions. A capability is evaluated against the current state through its preconditions and transforms that state through its effects.

2. Capability representation

Each capability is represented using the assignment's formal structure:

$$C_i = (T_i, I_i, O_i, P_i, E_i, K_i, R_i, Q_i, Rel_i, A_i, M_i)$$

Field	Meaning	Role in SCE
T	Capability type	Identity / mechanism block
I	Inputs	Output → input compatibility
O	Outputs	Output → input compatibility
P	Preconditions	Applicability and composition boundary
E	Effects	Effect → precondition compatibility
K	Constraints	Feasibility
R	Resources	Operational feasibility
Q	Cost / quality attributes	Operational behaviour
Rel	Reliability $\in [0,1]$	Operational quality
A	Availability	Operational quality
M	Execution mechanism	Implementation distinction

3. Token and field encoding

Let $b(t) \in R^{24}$ denote a deterministic token embedding. For any structured field F, the field representation is the mean of the embeddings of its tokens:

$$E(F) = (1 / |F|) \sum_{t \in F} b(t)$$

Structured values are tokenized in a way that preserves field content. A raw structured token is also included, reducing the chance that two different formal predicates collapse merely because they share individual words.

4. Capability embedding

The complete capability vector is a concatenation of semantically distinct blocks:

$$\phi_C(C_i) = [E_{id}, E_I, E_O, E_P, E_E, E_{R,K}, q_i]$$

The blocks correspond respectively to identity/type/mechanism, inputs, outputs, preconditions, effects, resources/constraints, and operational quality.

5. Operational-quality block

The operational vector is formed from normalized execution time, monetary cost, resource cost, risk, unreliability, and unavailability:

$$q_i = \text{normalize}([t/500, m/0.02, r/3, risk, 1-Rel, 1-A])$$

The normalization constants are dataset-scale choices for the supplied experiment. They are not intended as universal physical units.

6. Similarity

For encoded entities x and y, functional resemblance is measured by cosine similarity:

$$\text{sim}(x,y) = (\phi(x) \cdot \phi(y)) / (||\phi(x)|| \, ||\phi(y)||)$$

Cosine similarity is symmetric. It is therefore suitable for measuring resemblance, but it cannot by itself express the direction required by capability composition.

7. Directed capability compatibility

Let E_i be the effect block of C_i, P_j the precondition block of C_j, O_i the output block of C_i, and I_j the input block of C_j. We define:

$$\text{Comp}(C_i, C_j) = 0.6 \cos(E_i, P_j) + 0.4 \cos(O_i, I_j)$$

This score is explicitly directed: $\text{Comp}(C_i,C_j)$ need not equal $\text{Comp}(C_j,C_i)$. The 0.60 threshold used in the experiments is an operating point for this dataset, not a universal theoretical threshold.

8. Composition operator

For a valid sequence $C_1 \rightarrow C_2 \rightarrow \ldots \rightarrow C_n$, the composite is encoded using the same capability representation. The first capability contributes the external input and initial preconditions; the last contributes the external output. Effects, constraints, and resources are unioned.

Execution time, monetary cost, and resource cost are summed. Reliability is combined multiplicatively and availability is taken as the minimum availability across the chain.

$$C_{1:n} = C_n \blacksquare \ldots \blacksquare C_2 \blacksquare C_1$$

This gives a common representation for both atomic and composite capabilities.

9. Goal relevance

Goal relevance is treated as a ranking signal rather than a proof of goal achievement:

$$\text{GoalRel}(C,G) = \cos(E(\text{effects}(C)), E(\text{conditions}(G)))$$

A high value means the capability's effects resemble the desired goal conditions. A complete goal may still require several capabilities and explicit compatibility checks.

10. Design rationale

• Identity and mechanism are represented explicitly so alternative implementations remain distinguishable.

• Inputs/outputs and preconditions/effects receive separate blocks because they represent different kinds of compatibility.

• Operational attributes are explicit rather than silently conflated with semantic similarity.

• Composition uses a directed boundary score, avoiding the common mistake of equating semantic similarity with executability.

• The same encoder represents atomic and composite capabilities, allowing composition to be investigated in one vector space.